

COMICCRAFT

AI Comic Story Creator using Gemini Models

PROJECT DESIGN PHASE

System, application, interface, AI integration, routing, and output design based on the documented ComicCraft project.

1. Design Phase Overview

The design phase defines how ComicCraft is organized as a web-based AI comic-generation application. The documented design separates the system into three primary components: Frontend, Backend, and AI Integration. The frontend collects user preferences and displays the generated comic, the FastAPI backend manages routing and processing, and the AI services generate the story content and illustrations.

Design objective: provide a clear flow from creative input to AI generation, comic layout, browser preview, and downloadable PDF output.

2. High-Level System Architecture

Layer	Design / Technology	Main responsibility
Frontend	HTML, CSS, Jinja2	Collect user inputs and display generated comic content
Backend	FastAPI	Handle requests, routing, validation, AI workflow, and export
Story AI	Gemini Flash + Gemini Pro	Generate structured outlines, narration, and dialogue
Image AI	Stable Diffusion + Hugging Face Diffusers	Generate comic-style illustrations
Layout	layout_builder.py	Match images with corresponding panel stories
Export	FPDF / exporters.py	Compile comic content into a PDF

3. Frontend Design

The frontend is designed using HTML, CSS, and Jinja2 templates. The interface is intended to provide a simple and interactive path from input collection to comic preview and export confirmation.

index.html

Main input page where the user provides the story prompt, character name, setting, story tone, and art style.

comic_preview.html

Displays the generated comic panel-by-panel, including panel title, image, scene description, captions, and narration.

export_success.html

Displays a confirmation after the comic PDF is successfully downloaded and provides an option to create another comic.

4. User Input Design

Input	Purpose
Story Prompt	Main idea or concept for the comic
Character Name	Name of the main character / hero
Setting	Location in which the story occurs
Story Tone	Mood such as dramatic, funny, or poetic
Art Style	Visual preference such as anime, pixel art, comic book, or realistic

5. Backend Design

FastAPI is used as the backend framework. The backend receives form data or JSON requests, validates inputs, coordinates the AI-generation pipeline, builds the comic layout, exports the PDF, and renders the appropriate Jinja2 template or API response.

- Receive form-based or JSON-based user input.
- Validate JSON input using the documented Pydantic PromptRequest schema.
- Call the AI generation functions in the required sequence.
- Process and organize generated text and images.
- Pass the resulting layout to the frontend templates.
- Compile the comic into a downloadable PDF.
- Handle unexpected failures through HTTPException and try-except logic.

6. Route Design

Route	Design purpose
/	Loads index.html and provides the main comic input form.
/generate	Processes form submission and generates the comic preview.
/generate-comic/json	Accepts JSON input and returns comic layout data and PDF path.
/export-success	Displays the export confirmation page.
/test-image	Provides a developer route for testing image generation independently.

7. AI Integration Design

The AI layer is divided according to the type of content being generated. Gemini Flash is used for the structured five-panel outline, Gemini Pro expands that outline into detailed narration and character dialogue, and Stable Diffusion generates the visual illustration for each panel.

Stage	Module / function	Design output
Outline generation	gemini_flash.py / generate_outline()	Five-panel structured outline
Story generation	gemini_pro.py / generate_story()	Narration and character dialogue
Image generation	image_generator.py / generate_image()	Illustration for each panel

8. End-to-End Data Flow Design

1. User Input

User enters prompt, character, setting, tone, and art style.

2. FastAPI Request

The /generate route receives the form data.

3. Outline

generate_outline() creates the structured five-panel plan.

4. Story

`generate_story()` creates narration and dialogue.

5. Images

`generate_image()` creates an illustration for each panel.

6. Layout

`build_comic_layout()` associates images with panel text.

7. Preview

`comic_preview.html` renders the generated comic.

8. Export

`save_pdf()` compiles the comic using FPDF.

9. Confirmation

`export_success.html` confirms completion.

9. Comic Panel Design

The project design uses a five-panel structure. Each panel is designed to contain a panel number, title, scene description, image-generation prompt, generated illustration, and story content. The preview page presents these elements sequentially so the user can follow the narrative.

Panel element	Purpose
Panel number	Identifies the position of the panel in the story.
Panel title	Provides a short descriptive label.
Scene description	Sets the atmosphere and context of the scene.
Image prompt	Describes the visual content to be generated.
Comic image	Provides the generated visual illustration.
Caption / narration	Communicates environment, actions, emotions, and dialogue.

10. Layout and PDF Export Design

The layout design is handled by `layout_builder.py`. The `build_comic_layout()` function organizes the generated images and full comic story into a structured list, matching each image with its corresponding panel story. The `save_pdf()` function in `exporters.py` then uses FPDF to place the panel image and narration onto consecutive pages and saves the final PDF in the documented `static/exports` location.

11. Template and Dynamic Rendering Design

Jinja2Templates

FastAPI uses Jinja2Templates with the `templates` directory for dynamic HTML rendering.

index.html

Receives user inputs and submits them through POST to `/generate`.

comic_preview.html

Loops through the layout data and displays panel number, title, image, scene description, captions, and narration.

export_success.html

Displays the export confirmation and receives the exported PDF path as template context.

12. Design for Local Deployment

The documented deployment design uses a Python virtual environment, project dependencies installed from `requirements.txt`, and environment variables for sensitive API credentials. The FastAPI application is launched with Uvicorn and can be accessed locally through the browser.

Design item	Documented approach
Environment	Python virtual environment

Design item	Documented approach
Dependencies	Install packages from requirements.txt
Credentials	Store Gemini API key and Hugging Face API token in environment variables or .env
Server	Uvicorn running the FastAPI application
Application URL	http://127.0.0.1:8000
API documentation	http://127.0.0.1:8000/docs

13. Design Phase Checklist

- ✓ Three-layer architecture defined: frontend, backend, and AI integration.
- ✓ User input fields and interface flow defined.
- ✓ FastAPI backend responsibilities defined.
- ✓ Core application routes defined.
- ✓ AI model responsibilities separated by generation task.
- ✓ Five-panel comic structure defined.
- ✓ Dynamic Jinja2 templates defined.
- ✓ Comic layout and PDF export process defined.
- ✓ Local deployment structure defined.
- ✓ API testing interface defined through FastAPI documentation.

14. Project Design Phase Conclusion

The ComicCraft design phase establishes a modular architecture in which the frontend, FastAPI backend, AI generation services, layout builder, and PDF exporter work together as a single comic-generation workflow. The design supports both browser-based form interaction and JSON-based API generation while maintaining a clear path from user input to generated comic preview and final PDF.

Source basis: This document is derived from the uploaded ComicCraft project documentation. It preserves the documented architecture, modules, routes, templates, workflow, and deployment approach rather than introducing unsupported implementation details.